Since we run minikube on top of Docker, remember to start the Docker Engine first. Then make sure the default cluster is not running by executing the `minikube stop` command. From now on, we won't use the default cluster. Instead, we'll create a custom one for working with Polar Bookshop. With `minikube` you can create and control multiple clusters identified via *profiles*. When no profile is specified, `minikube` falls back on the default cluster.

WARNING Running the examples on a local Kubernetes cluster requires Docker to have at least 2 CPUs and 4 GB of memory. If you are using Docker Desktop for Mac or for Windows, and you need to increase the resources assigned to the Docker Engine, refer to the product documentation for instructions on how to do that for your specific operating system (https://docs.docker.com/desktop).

Let's create a new Kubernetes cluster named `polar` on top of Docker. This time, we also want to declare the resource limits for CPU and memory:

```
$ minikube start --cpus 2 --memory 4g --driver docker --profile polar
```

You can get a list of all the nodes in the cluster with the following command:

```
$ kubectl get nodes

NAME    STATUS   ROLES                  AGE   VERSION
polar   Ready    control-plane,master   21s   v1.24.3
```

The cluster we have just created is composed of a single node, which hosts the Control Plane and acts as a worker node for deploying containerized workloads.

You can use the same Kubernetes client (`kubectl`) to interact with different local or remote clusters. The following command will list all the available *contexts* with which you can interact: